

1. Introduction

In programming languages, variables and objects occupy memory during program execution. Two important concepts related to memory management are lifetime and garbage collection.

While scope defines *where* a variable can be accessed, lifetime defines *how long* the variable exists in memory.

Garbage collection is a mechanism used to automatically reclaim unused memory, especially for heap-dynamic variables.

2. Lifetime

Lifetime of a variable is defined as the time duration during which the variable is bound to a memory location during program execution.

In simple words, lifetime answers:

“For how long does this variable exist in memory?”

Lifetime is a runtime concept and is independent of scope.

3. Importance of Lifetime

- Efficient use of memory
- Avoids memory wastage
- Supports recursion and dynamic data structures
- Helps in understanding memory-related errors such as dangling pointers and memory leaks

4. Categories of Variable Lifetime

4.1 Static Lifetime

- Variables exist throughout the entire execution of the program
- Memory allocated before program execution and released after program termination

Examples:

- Global variables
- **Static variables**

Example:

```
static int x;
```

Advantages:

- Fast access
- Simple implementation

Disadvantages:

- Memory occupied even when variable is not used

4.2 Stack-Dynamic Lifetime

- Variables are created when a function or block is entered
- Destroyed when the function or block exits
- Memory allocated on the runtime stack

Examples:

- Local variables
- Function parameters

Example:

```
void fun()
{
    int y; // stack-dynamic variable
}
```

Advantages:

- Supports recursion
- Automatic allocation and deallocation

Disadvantages:

- Size must be known at compile time

4.3 Explicit Heap-Dynamic Lifetime

- Variables allocated and deallocated explicitly by the programmer
- Memory taken from the heap

Examples:

- malloc() and free() in C
- new and delete in C++

Example:

```
int *p = (int*)malloc(sizeof(int));
free(p);
```

Advantages:

- Flexible memory usage
- Suitable for dynamic data structures

Disadvantages:

- Programmer errors may cause:
 - Memory leaks
 - Dangling pointers
-

4.4 Implicit Heap-Dynamic Lifetime

- Variables allocated automatically at runtime
- Deallocation handled by garbage collector
- Common in modern high-level languages

Examples:

- Objects in Java, Python, C#

Example:

Object obj = new Object();

Advantages:

- Safe and simple memory management
- No explicit deallocation required

Disadvantages:

- Garbage collection overhead

5. Garbage Collection

Garbage collection is the automatic process of reclaiming heap memory that is no longer reachable or usable by the program.

Garbage refers to allocated memory that cannot be accessed by any active reference.

Garbage collection is mainly used for heap-dynamic variables.

6. Need for Garbage Collection

- Prevents memory leaks
- Avoids dangling references
- Reduces programmer burden

- Improves reliability and safety of programs

7. Garbage Collection Techniques

7.1 Reference Counting

- Each object maintains a count of references
- Count incremented when reference is created
- Count decremented when reference is destroyed
- Object is reclaimed when count becomes zero

Advantages:

- Simple to implement

Disadvantages:

- Cannot handle circular references

7.2 Mark-and-Sweep

Steps:

1. Mark phase – all reachable objects are marked
2. Sweep phase – unmarked objects are reclaimed

Advantages:

- Handles circular references

Disadvantages:

- Program execution may pause during collection

7.3 Copying Garbage Collection

- Heap divided into two regions
- Live objects copied to the new region
- Old region cleared completely

Advantages:

- Fast allocation
- No fragmentation

Disadvantages:

- Requires extra memory

8. Problems Without Garbage Collection

- Memory leaks
- Dangling pointers
- Program crashes
- Reduced system performance

9. Relationship Between Lifetime and Garbage Collection

- Heap-dynamic variables have unpredictable lifetimes
- Garbage collection automatically terminates the lifetime of unreachable objects
- Ensures safe and efficient memory usage

10. Conclusion

Lifetime defines the existence duration of variables in memory, while garbage collection provides an automatic mechanism to reclaim unused heap memory.

Modern programming languages widely adopt garbage collection to improve safety, reliability, and ease of programming, making memory management less error-prone.